U.T. 4: Normalización de Bases de Datos

Teoría, Anomalías y Formas Normales

Departamento de Informática - IES Francisco de Goya

febrero de 2026

1. Introducción y Necesidad

¿Por qué Normalizar?

Al diseñar una base de datos (pasando de E/R a Relacional o directamente), podemos cometer errores de diseño.

La **Normalización** es una técnica formal para organizar los datos que nos ayuda a:

- Detectar errores en el diseño.
- Corregir estructuras defectuosas.
- Asegurar propiedades deseables en el esquema.

Principio Básico del Diseño

"Hechos distintos se deben almacenar en objetos distintos"

Ejemplo: Mala Normalización

Problema: Una tabla con redundancias (Autor, Nacionalidad, Libro... todo junto).

Autor	Nacional.	ISBN	Título	Editorial	Año
Pérez Martín	Española	7445	Bases de Datos	Mc. Graw-Hill	1998
Pérez Martín	Española	8458	Lenguaje C++	Mc. Graw-Hill	1996
Pérez Martín	Española	5885	El Modelo Relacional	Ra-Ma	1995
García Sánchez	Española	5885	El Modelo Relacional	Ra-Ma	1995
García Sánchez	Española	2169	Linux	Anaya	1999
Cousteau	Francesa	2169	Linux	Anaya	1999
Romboni	Italiana	2169	Linux	Anaya	1999

Esto provoca anomalías de inserción, modificación y borrado. ¿Qué cosas pueden salir mal?

Problemas de un mal diseño 1/2

- **Anomalías de inserción:** Dar de alta a un libro obliga a insertar tantas tuplas como autores tenga el libro. Además, pueden añadirse inconsistencias. ¿Qué pasa si al añadir una de esas tuplas pongo un año diferente para el mismo libro?
- **Anomalías de modificación:** Cambiar la editorial de un libro obliga a modificar todas las tuplas que corresponden a ese libro.
- **Anomalías de borrado:** Para dar de baja a un libro hay que borrar tantas tuplas como autores tenga el libro y, viceversa, para dar de baja a un autor hay que borrar tantas tuplas como libros haya escrito.

Problemas de un mal diseño 2/2

- **Pérdida de semántica:** No podemos almacenar información sobre un autor sobre el que no exista ningún libro en la biblioteca. Esto es debido a que el ISBN forma parte de la clave, y no puede contener valores nulos (integridad de la clave).
  - A nálogamente, no podemos registrar libros con autores anónimos. Por la misma razón que antes, dado que el nombre del autor también forma parte de la clave primaria.
- **Pérdidas de información:** Al dar de baja a un libro, se pierde información sobre sus autores (si éstos sólo tuviesen ese libro en la BD).
  - A nálogamente, al dar de baja a un autor se pierde información sobre los libros que haya escrito el sólo.

Solución al Ejemplo 1

Si aplicamos un buen diseño, separamos las entidades:

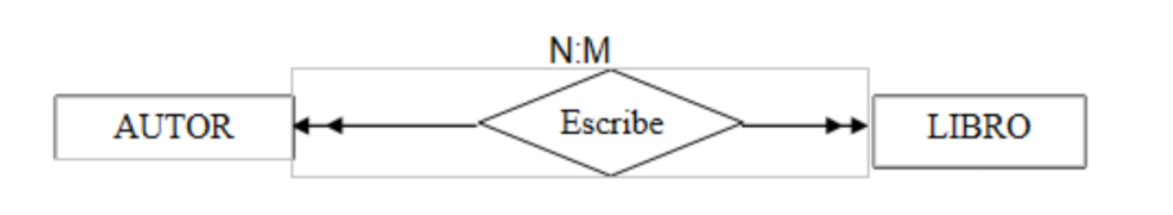


Figura 1: Esquema Relacional Correcto: Autor, Libro y Escribe

Solución al Ejemplo 1

Tabla: Libro

ISBN	Título	Editorial	Año
7445	Bases de Datos	Mc. Graw-Hill	1998
8458	Lenguaje C++	Mc. Graw-Hill	1996
5885	El Modelo Relacional	Ra-Ma	1995
2169	Linux	Anaya	1999

Tabla: Autor

Autor	Nacional.
Pérez Martín	Española
García Sánchez	Española
Cousteau	Francesa
Romboni	Italiana

Tabla: Libro_Autor

ISBN	Autor
7445	Pérez Martín
8458	Pérez Martín
5885	Pérez Martín
5885	García Sánchez
2169	García Sánchez
2169	Cousteau
2169	Romboni

2. Principios Generales

Tablas Relacionales

Código	Nombre	Posición	Salario
1	Edgardo Trujillo	Gerente	19000
2	Lidimarie Fonsi	Empleada	12000
3	Jean Piaget	Empleado	13500
4	Jerome Bruner	Empleado	14000

Figura 2: Estructura de una tabla relacional

La normalización distribuye campos en tablas relacionadas para:

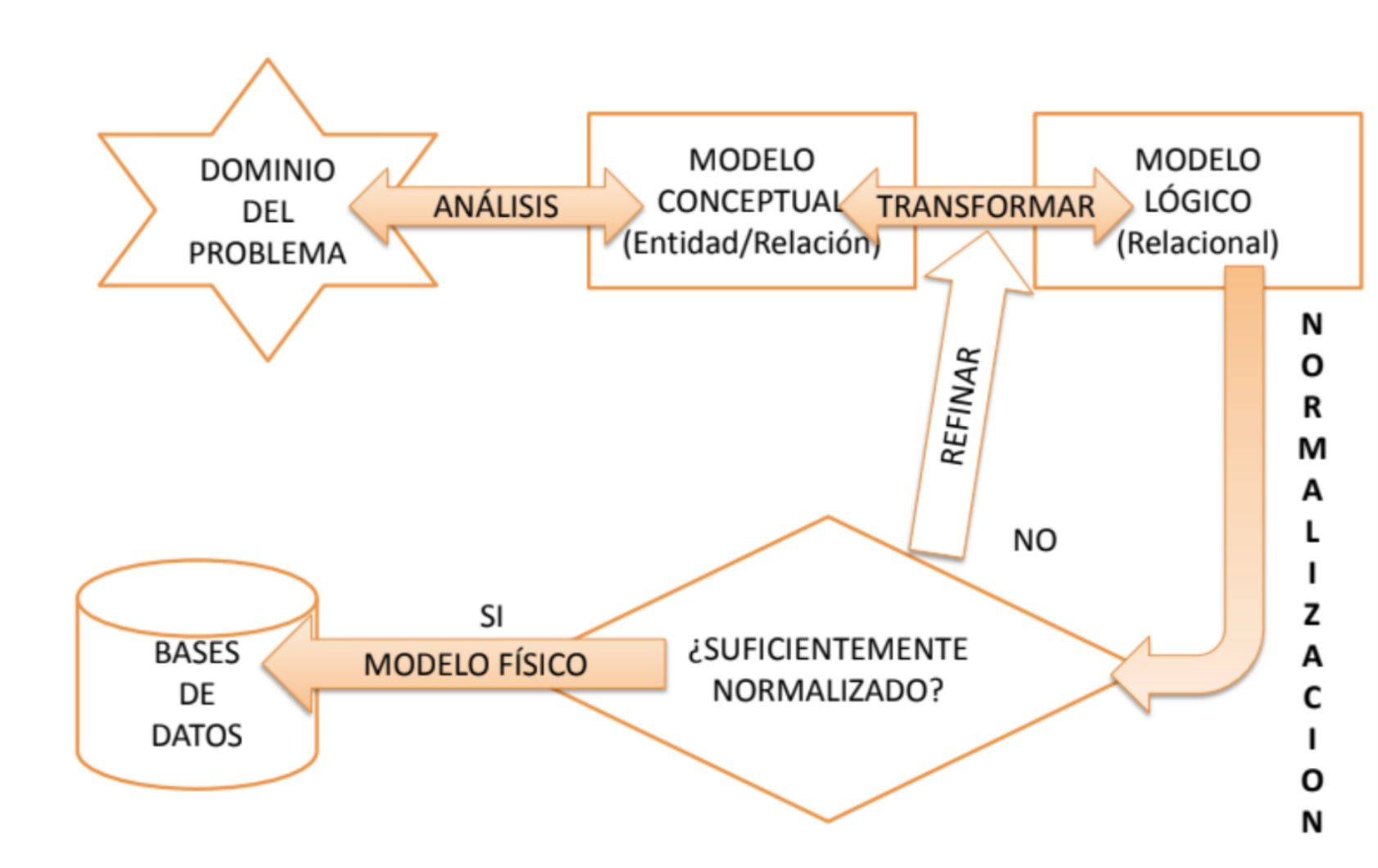
- Evitar redundancia.
- Proteger la integridad de datos.
- Facilitar el mantenimiento.

Reglas de Oro

Para evitar los problemas anteriores, aplicamos estas reglas:

- **Unicidad de información:** Cada campo debe contener un único tipo de información (no mezclar Nombre y Apellidos si se necesitan por separado).
- **Clave Principal (PK):** Cada tabla debe tener un identificador único.
- **Independencia:** Poder cambiar campos no clave sin afectar a otros.
- **Dependencia total:** Todos los datos de la tabla deben describir el "tema"de la tabla (la clave primaria) y nada más.

Reglas Fundamentales



3. Dependencias Funcionales

¿Qué es una Dependencia Funcional?

Dados dos atributos A y B, decimos que B depende funcionalmente de A ($A \rightarrow B$) si:

> *Para cada valor de A existe un valor de B, y solo uno, asociado con él.*

Ejemplo:

$$DNI \rightarrow Nombre$$

(Si sé el DNI, sé el Nombre. El nombre no puede variar para un mismo DNI).

Ejemplo: Préstamos de Biblioteca 1/2

Prestamos (Cód_Libro, Título, Editorial, Num_Socio, Nombre, Domicilio, FechaPréstamo, FechaDevolución)

Ejemplo: Préstamos de Biblioteca 2/2

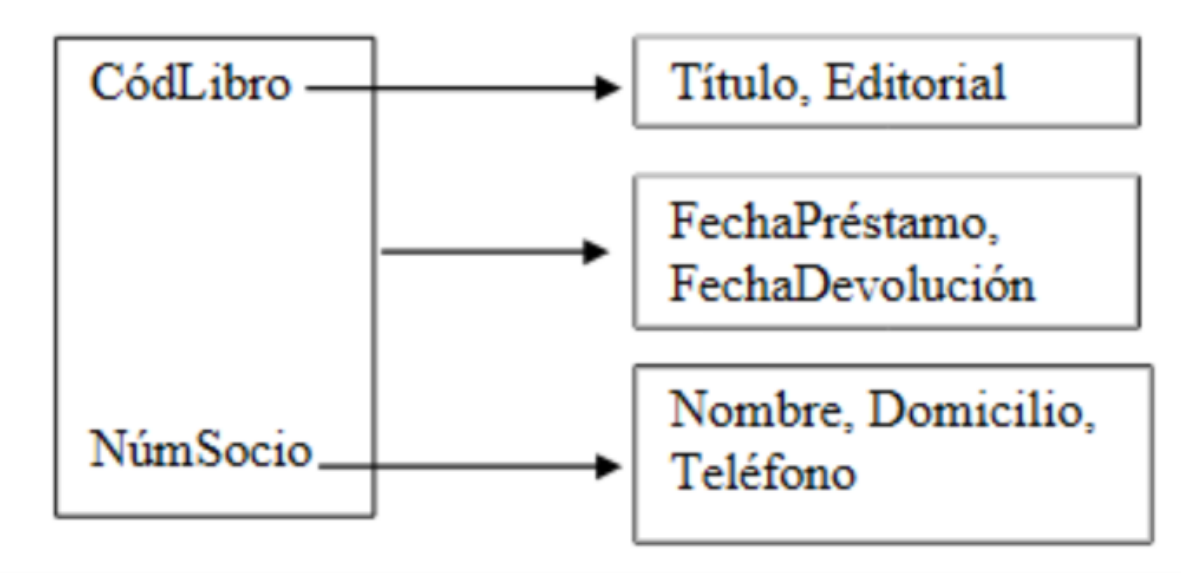


Figura 3: Diagrama de Dependencias Funcionales

- Cód_Libro $\rightarrow$ Título, Editorial.
- Num_Socio $\rightarrow$ Nombre, Domicilio.
- (Cód_Libro, Num_Socio) $\rightarrow$ FechaPréstamo, FechaDevolución

Dependencia Funcional Completa

Definición

Sea un atributo X compuesto (X_1, X_2). Se dice que Y tiene **dependencia funcional completa** respecto a X si depende de X en su totalidad, y no de X_1 ni de X_2 por separado.

$$X \Rightarrow Y$$

Ejemplo: ¿Dependencias funcionales completas de la PK?

`Prestamos (`<u>`Cód_Libro`</u>`, Título, Editorial, `<u>`Num_Socio`</u>`, Nombre, Domicilio, FechaPréstamo, FechaDevolución)`

Regla de Oro: En una tabla, todos los atributos deben tener una dependencia funcional completa de la clave principal.

Ejemplo: ¿Dependencias funcionales completas de la PK?

`Prestamos (Cód_Libro, Título, Editorial, Num_Socio, Nombre, Domicilio, FechaPréstamo, FechaDevolución)`

(Cód_Libro, Num_Socio) $\Rightarrow$ FechaPréstamo, FechaDevolución

(Cód_Libro, Num_Socio) $\not\Rightarrow$ Título

(Cód_Libro, Num_Socio) $\rightarrow$ Título

Regla de Oro: En una tabla, todos los atributos deben tener una dependencia funcional completa de la clave principal.

Dependencia Funcional Transitiva

Definición

Sean X, Y, Z atributos de una misma entidad. Se dice que Z depende transitivamente de X si:

$$X \to Y \quad \text{y} \quad Y \to Z$$

(Suponiendo que X no depende de Y).

Ejemplo: Permiso de Conducir

- **1.** *FechaNacimiento* $\to$ *Edad*
  (Sabiendo la fecha, sabemos la edad exacta).
- **2.** *Edad* $\to$ *Conducir*
  (La edad determina legalmente si se puede conducir).
- **Conclusión:** *FechaNacimiento* $\to$ *Conducir*
  (Indirectamente, la fecha de nacimiento determina si puedes conducir).

Tipos de Dependencia

Dependencia Funcional Completa

Un campo depende de **toda** la clave primaria, no solo de una parte.

Dependencia Transitiva

$$X \rightarrow Y \rightarrow Z$$

Z depende de X a través de Y.

4. Formas Normales

Primera Forma Normal (1FN)

Definición

Una tabla está en 1FN si **no admite atributos multivaluados**. Es decir, cada celda debe contener un único valor atómico.

Ejemplo de Violación (Tabla NO Normalizada):

NPed	Fecha	CodC	Nom	CodArt	Cant.	Desc.	Pre.
123	19/11	C14	Pepe	A03,A15	24,10	Lápiz,Goma	25,15
145	24/11	C67	Juan	A02,A15	1000,10	Folio,Goma	1,15

** Nota cómo los artículos están separados por comas en una misma celda.*

¿Cómo pasar a 1FN?

Para corregir la tabla anterior, seguimos estos pasos:

1. **Identificar** los atributos que se repiten (multivaluados) para una misma clave.
2. **Separar:** Crear una nueva tabla con esos atributos, eliminándolos de la original.
3. **Nueva Clave:** La clave principal de la nueva tabla será la concatenación de:

$$\text{Clave Original} + \text{Clave del Atributo Repetido}$$

Solución: Tablas Normalizadas

1. **Tabla PEDIDOS** (Datos únicos por pedido)

NPed (PK)	Fecha	Cod_Cli	Nom_Cli
123	19/11/96	C14	Pepe
145	24/11/96	C67	Juan

2. **Tabla PED_ART** (Desglose de artículos)

NPed (PK)	CodArt (PK)	Cant.	Desc.	Precio
123	A03	24	Lápiz	25
123	A15	10	Goma	15
145	A02	1000	Folio	1
145	A05	10	Goma	15

* Nota: La clave de la segunda tabla es compuesta (NPed + CodArt) para evitar duplicados. **
Pregunta: ¿Qué pasa aquí con las dependencias funcionales?

Segunda Forma Normal (2FN)

Definición

Una relación está en 2FN si:

- Está en **1FN**.
- Todos los atributos que no son clave dependen de **toda** la clave principal (Dependencia Funcional Completa).

¿Qué significa esto?

- No pueden existir **dependencias parciales** (atributos que dependan solo de una parte de la clave).
- *Nota:* Si la clave principal es simple (un solo campo), la tabla ya está automáticamente en 2FN (si cumple 1FN).

Ejemplo: Tabla NO Normalizada

Supongamos una tabla de notas.

Clave Principal (PK): `(COD_AL, COD_ASIG)`

COD_AL	NOM_AL	TF_AL	COD_ASIG	NOTA	CURSO	HORAS
1111	Pepe Pérez	911234567	LMSI	6	1ASIR	4
1111	Pepe Pérez	911234567	SSOO	7	1ASIR	8
2222	Ana López	912345678	LMSI	8	1ASIR	4
2222	Ana López	912345678	SSOO	5	1ASIR	8
2222	Ana López	912345678	REDES	6	1ASIR	7
3333	Juan Gil	913456789	FOL	7	2DAI	2
3333	Juan Gil	913456789	RET	5	2DAI	2

Problema: Hay redundancia. Si Ana cambia de teléfono, hay que actualizar 3 filas.

Análisis de Dependencias

Analizamos qué campos dependen de la clave (`COD_AL`, `COD_ASIG`):

1. **Datos del Alumno:**

$$\text{COD_AL} \rightarrow \text{NOM_AL, TF_AL}$$

   X Dependencia Parcial (Solo depende de una parte de la clave).

2. **Datos de la Asignatura:**

$$\text{COD_ASIG} \rightarrow \text{CURSO, HORAS}$$

   X Dependencia Parcial (Solo depende de la otra parte).

3. **La Nota:**

$$(\text{COD_AL, COD_ASIG}) \rightarrow \text{NOTA}$$

   ✓Dependencia Completa (Depende de ambos: quién y en qué).

Solución: Tablas en 2FN

Separamos las dependencias parciales en nuevas tablas.

1. ALUMNO

COD	NOM	TF
1111	Pepe	911...
2222	Ana	912...
3333	Juan	913...

2. ASIGNATURA

COD	CUR	HRS
LMSI	1ASIR	4
SSOO	1ASIR	8
REDES	1ASIR	7
FOL	2DAI	2
RET	2DAI	2

3. NOTAS (Relación)

ALU	ASIG	NOTA
1111	LMSI	6
1111	SSOO	7
2222	LMSI	8
2222	SSOO	5
2222	REDES	6
3333	FOL	7
3333	RET	5

Ejercicio: ¿Está en 2FN?

1. **Tabla PEDIDOS** (Datos únicos por pedido)

NPed (PK)	Fecha	Cod_Cli	Nom_Cli
123	19/11/96	C14	Pepe
145	24/11/96	C67	Juan

2. **Tabla PED_ART** (Desglose de artículos)

NPed (PK)	CodArt (PK)	Cant.	Desc.	Precio
123	A03	24	Lápiz	25
123	A15	10	Goma	15
145	A02	1000	Folio	1
145	A05	10	Goma	15

Solución en pizarra.

Tercera Forma Normal (3FN)

Definición

Una relación está en 3FN si:

- Ya cumple la **2FN**.
- **No existen dependencias transitivas.**

Regla Simplificada:

> *"Los atributos no clave deben depender únicamente de la clave, y no de otros atributos que no sean clave."*

Si un atributo depende de otro campo que no es la clave principal, tenemos una **Dependencia Transitiva** que debemos eliminar.

Ejemplo: Tabla con Dependencia Transitiva

Supongamos una tabla de Clientes.

Clave Principal (PK): `DNI_CL`

DNI_CL	NOM_CL	DIR_CL	C_POS	PROV
1111	Pepe Pérez	Gran Vía 17	28005	Madrid
2222	Juan López	Galileo 23	28015	Madrid
3333	Ana Gómez	Turia 45	46007	Valencia
4444	Marta Gil	Naval 29	00817	Barcelona
5555	Laura Mas	Aviación 14	46027	Valencia

Problema: Aunque la Provincia está relacionada con el Cliente (DNI_CL $\rightarrow$ Provincia), en realidad depende directamente del Código Postal (DNI_CL $\rightarrow$ C_POS $\rightarrow$ PROV.)

Análisis de Dependencias (3FN)

Analizamos las relaciones entre los datos:

1. **Dependencia Directa (Correcta):**

$$\text{DNI_CL} \rightarrow \text{C_POS}$$

   (El DNI determina dónde vive el cliente y su código postal).

2. **Dependencia Transitiva (Incorrecta):**

$$\text{C_POS} \rightarrow \text{PROV}$$

   (La provincia depende del Código Postal, no directamente del DNI del cliente).

Cadena transitiva:

$$\text{DNI_CL} \rightarrow \text{C_POS} \rightarrow \text{PROV}$$

Debemos romper esta cadena separando Código Postal de

Solución: Tablas en 3FN

Extraemos la dependencia transitiva a una nueva tabla de referencia.

1. CLIENTES

DNI	NOM	DIR	CP
1111	Pepe	Gran Vía	28005
2222	Juan	Galileo	28015
3333	Ana	Turia	46007
4444	Marta	Naval	00817

2. PROVINCIAS

CP (PK)	PROV
28005	Madrid
28015	Madrid
46007	Valencia
00817	Barcelona
46027	Valencia

** Ahora PROV depende únicamente de la clave (CP) de su propia tabla.*

Resumen

La base de datos está bien diseñada si:

1. Cada celda tiene un solo valor **(1FN)**.
2. Todo depende de la clave, toda la clave **(2FN)**...
3. ...y nada más que la clave **(3FN)**.